Relative path: dashboard/index.html
Filename: index.html Extension: .html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>OASIS — Analyst Dashboard</title>
  <style>
    :root {
      color-scheme: dark;
      --bg: #081018;
      --panel: #101b26;
      --panel-2: #0d1620;
      --line: rgba(148, 163, 184, 0.16);
      --text: #e5eef8;
      --muted: #91a4b8;
      --accent: #79d9ff;
      --accent-2: #7ef0b8;
      --warn: #f7c76d;
      --danger: #ff7e8a;
      --shadow: 0 18px 48px rgba(0, 0, 0, 0.35);
    }

    * { box-sizing: border-box; }
    body {
      margin: 0;
      color: var(--text);
      background:
        radial-gradient(circle at top left, rgba(121, 217, 255, 0.16), transparent 30%),
        radial-gradient(circle at top right, rgba(126, 240, 184, 0.12), transparent 26%),
        linear-gradient(180deg, #081018 0%, #09131d 52%, #071018 100%);
      font-family: "Aptos", "Trebuchet MS", "Segoe UI", sans-serif;
    }

    header {
      padding: 22px 28px 16px;
      border-bottom: 1px solid var(--line);
      background: rgba(8, 16, 24, 0.82);
      backdrop-filter: blur(14px);
      display: flex;
      justify-content: space-between;
      align-items: flex-end;
      gap: 20px;
      position: sticky;
      top: 0;
      z-index: 10;
    }

    .title h1 {
      margin: 0;
      font-size: 22px;
      letter-spacing: 0.02em;
    }

    .title p {
      margin: 8px 0 0;
      color: var(--muted);
      font-size: 13px;
      max-width: 900px;
      line-height: 1.45;
    }

    #meta {
      font-size: 13px;
      color: var(--muted);
      text-align: right;
      white-space: nowrap;
    }

    .summary {
      display: grid;
      grid-template-columns: repeat(5, minmax(0, 1fr));
      gap: 12px;
      padding: 14px 28px 0;
```

```
}

.summary-card {
  border: 1px solid var(--line);
  background: linear-gradient(180deg, rgba(16, 27, 38, 0.98), rgba(10, 18, 28, 0.98));
  border-radius: 18px;
  box-shadow: var(--shadow);
  padding: 14px 16px 13px;
  min-height: 110px;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  animation: rise 220ms ease-out;
}

.summary-label {
  color: var(--muted);
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 0.12em;
}

.summary-value {
  font-size: 26px;
  line-height: 1;
  margin-top: 8px;
  font-weight: 700;
}

.summary-note {
  margin-top: 8px;
  color: #b9c8d8;
  font-size: 12px;
  line-height: 1.35;
}

.summary-ok { color: var(--accent-2); }
.summary-warn { color: var(--warn); }

.layout {
  display: grid;
  grid-template-columns: 46% 54%;
  min-height: calc(100vh - 180px);
  padding: 14px 18px 20px;
  gap: 14px;
}

#queue,
#detail {
  border: 1px solid var(--line);
  background: rgba(16, 27, 38, 0.88);
  border-radius: 20px;
  box-shadow: var(--shadow);
  overflow: hidden;
}

#queue {
  overflow-y: auto;
}

#detail {
  overflow-y: auto;
  padding: 20px;
}

table { width: 100%; border-collapse: collapse; }
thead th {
  position: sticky;
  top: 0;
  background: rgba(13, 22, 32, 0.98);
  z-index: 1;
  color: var(--muted);
  font-size: 11px;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  text-align: left;
```

```
  padding: 14px 14px 12px;
  border-bottom: 1px solid var(--line);
}

tbody td {
  padding: 14px;
  border-bottom: 1px solid rgba(148, 163, 184, 0.1);
  font-size: 13px;
  vertical-align: top;
}

tbody tr { cursor: pointer; transition: background 120ms ease; }
tbody tr:hover { background: rgba(121, 217, 255, 0.06); }
tbody tr.selected { background: rgba(121, 217, 255, 0.09); }

.queue-entity {
  display: grid;
  gap: 4px;
}

.queue-entity strong { font-size: 14px; }
.queue-subtle { color: var(--muted); font-size: 12px; }
.score { font-variant-numeric: tabular-nums; font-size: 15px; font-weight: 700; }

.badge {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 4px 10px;
  border-radius: 999px;
  font-size: 11px;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.08em;
}

.escalate { background: rgba(255, 126, 138, 0.16); color: #ff9aa3; }
.isolate { background: rgba(247, 199, 109, 0.16); color: #ffd98b; }
.acknowledge { background: rgba(126, 240, 184, 0.16); color: #9ef3c7; }

#detail h2 {
  margin: 0 0 6px;
  font-size: 24px;
}

.detail-kicker {
  color: var(--muted);
  font-size: 13px;
  margin-bottom: 20px;
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 14px;
}

.field {
  border: 1px solid var(--line);
  background: rgba(8, 16, 24, 0.58);
  border-radius: 16px;
  padding: 14px;
}

.field label {
  display: block;
  font-size: 11px;
  color: var(--muted);
  text-transform: uppercase;
  letter-spacing: 0.1em;
  margin-bottom: 8px;
}

.field .val { font-size: 14px; line-height: 1.45; }
.field.compact { min-height: 100%; }
```

```
.chips {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.chip {
  background: rgba(148, 163, 184, 0.1);
  border: 1px solid rgba(148, 163, 184, 0.12);
  padding: 8px 10px;
  border-radius: 12px;
  font-size: 12px;
  line-height: 1.3;
  max-width: 100%;
}

.chip .raw {
  display: block;
  color: var(--muted);
  font-size: 10px;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  margin-bottom: 3px;
}

.chip .explain {
  color: var(--text);
  font-size: 12px;
}

.entity-meta {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
}

.metric-tile {
  background: rgba(121, 217, 255, 0.06);
  border: 1px solid rgba(121, 217, 255, 0.14);
  border-radius: 12px;
  padding: 10px;
}

.metric-tile .k {
  color: var(--muted);
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  margin-bottom: 4px;
}

.metric-tile .v {
  font-size: 15px;
  font-weight: 700;
}

.actions {
  display: flex;
  gap: 10px;
  margin-top: 14px;
}

.actions button {
  flex: 1;
  padding: 12px 14px;
  border: 1px solid rgba(148, 163, 184, 0.18);
  border-radius: 12px;
  font-weight: 700;
  cursor: pointer;
  font-size: 13px;
  color: #fff;
}

.btn-ack { background: #21262d; color: #e6e6e6; }
.btn-iso { background: #7d5a00; color: #fff; }
```

```
    .btn-esc { background: #8e1519; color: #fff; }
    .btn-active { outline: 2px solid var(--accent); }

    .empty {
      padding: 44px 24px;
      color: var(--muted);
      text-align: center;
      border: 1px dashed rgba(148, 163, 184, 0.18);
      border-radius: 18px;
      background: rgba(8, 16, 24, 0.35);
    }

    .grounded-ok { color: var(--accent-2); font-weight: 700; }
    .grounded-bad { color: var(--danger); font-weight: 700; }

    @keyframes rise {
      from { transform: translateY(8px); opacity: 0; }
      to { transform: translateY(0); opacity: 1; }
    }

    @media (max-width: 1180px) {
      .summary { grid-template-columns: repeat(2, minmax(0, 1fr)); }
      .layout { grid-template-columns: 1fr; }
      header { align-items: flex-start; flex-direction: column; }
      #meta { text-align: left; }
    }

    @media (max-width: 720px) {
      .summary { grid-template-columns: 1fr; }
      .grid, .entity-meta { grid-template-columns: 1fr; }
      .actions { flex-direction: column; }
    }
  </style>
</head>
<body>
<header>
  <div class="title">
    <h1>OASIS — Analyst Dashboard</h1>
    <p>Ranked alerts, grounded narratives, and entity context in one view. The summary strip is the quicke
st read; the detail pane explains why each alert rose to the top.</p>
  </div>
  <div id="meta">loading…</div>
</header>

<section id="summary" class="summary"></section>

<div class="layout">
  <div id="queue">
    <table>
      <thead>
        <tr>
          <th>Impact</th>
          <th>Entity</th>
          <th>Type</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody id="queue-body"></tbody>
    </table>
  </div>
  <div id="detail"><div class="empty">Select an alert from the queue to see entity history, risk score, an
d contributing factors.</div></div>
</div>

<script>
let incidents = [];
let selected = null;
let dashboardStats = null;

const FEATURE_PHRASES = {
  hour_of_day: "Login occurred outside the learned schedule.",
  session_duration_sec: "Session duration deviated from the baseline.",
  num_resources_touched: "The session touched more resources than usual.",
  num_new_resources: "The session accessed previously unseen resources.",
  auth_failures: "Authentication failures spiked during the session.",
  num_distinct_geo: "The session spanned multiple geographies.",
```

```
  is_new_geo: "Activity came from a new geographic location.",
  is_new_device: "A new device fingerprint appeared.",
  sequence_length: "The action sequence length was unusual.",
  off_hours: "Activity happened outside normal operating hours.",
  impossible_travel_velocity: "Geo movement was too fast to be plausible.",
  "action:login": "The session included repeated login actions.",
  "action:read_sensor": "The session read sensor data.",
  "action:write_actuator": "The session wrote to an actuator.",
  "action:api_call_write": "The session performed write-oriented API calls.",
  "action:grant_admin": "The session attempted administrative privilege changes.",
  "action:access_admin_panel": "The session touched an admin-only control path.",
  "action:modify_permissions": "The session changed permission settings.",
  "action:access_root_shell": "The session reached a root-level shell action.",
  "action:disable_logging": "The session attempted to disable logging.",
};

const ZONE_LABELS = {
  user: "enterprise",
  service_account: "data_center",
  edge_device: "ot_edge",
};

function formatPercent(value) {
  return `${value.toFixed(1)}%`;
}

function describeFeature(feature) {
  return FEATURE_PHRASES[feature] || feature.replace(/_/g, " ");
}

function entityHistory(entityId) {
  return incidents.filter(r => r.entity_id === entityId);
}

function renderSummary() {
  const summary = document.getElementById('summary');
  if (!dashboardStats) {
    summary.innerHTML = '';
    return;
  }

  const cards = [
    {
      label: 'Critical Incidents',
      value: dashboardStats.critical_incidents,
      note: `${dashboardStats.flagged_sessions} flagged sessions in the ranked queue`,
      tone: 'summary-warn',
    },
    {
      label: 'Top Alert Precision',
      value: formatPercent(dashboardStats.top_alert_precision * 100),
      note: 'precision at the top 1% alert budget',
      tone: 'summary-ok',
    },
    {
      label: 'Grounded Narratives',
      value: formatPercent(dashboardStats.grounded_narratives_pct),
      note: `${dashboardStats.grounded_narratives_passed}/${dashboardStats.grounded_narratives_total} pass
ed the groundedness check`,
      tone: dashboardStats.grounded_narratives_pct >= 99 ? 'summary-ok' : 'summary-warn',
    },
    {
      label: 'Concept Drift',
      value: dashboardStats.concept_drift ? '✓' : '✕',
      note: dashboardStats.concept_drift ? 'insider drift adapted successfully' : 'drift adaptation missin
g',
      tone: dashboardStats.concept_drift ? 'summary-ok' : 'summary-warn',
    },
    {
      label: 'Cold Start',
      value: dashboardStats.cold_start ? '✓' : '✕',
      note: dashboardStats.cold_start ? 'warm-up logic is active in the detector' : 'cold-start handling m
issing',
      tone: dashboardStats.cold_start ? 'summary-ok' : 'summary-warn',
    },
  ];
```

```
  summary.innerHTML = cards.map(card => `
    <article class="summary-card">
      <div class="summary-label">${card.label}</div>
      <div class="summary-value ${card.tone}">${card.value}</div>
      <div class="summary-note">${card.note}</div>
    </article>
  `).join('');
}

async function load() {
  const [incidentsRes, statsRes] = await Promise.all([
    fetch('/api/incidents'),
    fetch('/api/dashboard-stats'),
  ]);

  incidents = await incidentsRes.json();
  dashboardStats = await statsRes.json();

  document.getElementById('meta').textContent =
    `${incidents.length} flagged sessions · ${formatPercent(dashboardStats.top_alert_precision * 100)} top
-alert precision · ${formatPercent(dashboardStats.grounded_narratives_pct)} grounded narratives`;

  renderSummary();
  renderQueue();
}

function renderQueue() {
  const body = document.getElementById('queue-body');
  body.innerHTML = '';

  incidents.forEach((r) => {
    const tr = document.createElement('tr');
    tr.className = r.session_id === selected ? 'selected' : '';
    tr.onclick = () => {
      selected = r.session_id;
      renderQueue();
      renderDetail(r);
    };

    const entityContext = r.entity_context || {};
    tr.innerHTML = `
      <td class="score">${r.impact_score.toFixed(2)}</td>
      <td>
        <div class="queue-entity">
          <strong>${r.entity_id}</strong>
          <span class="queue-subtle">${entityContext.asset_type || r.asset_type || 'unknown'} · ${entityCo
ntext.zone || r.zone || 'unknown'} · ${entityContext.alert_count ?? r.alert_count ?? 0} alerts</span>
        </div>
      </td>
      <td>${r.anomaly_type}</td>
      <td><span class="badge ${r.recommended_action}">${r.recommended_action}</span></td>
    `;
    body.appendChild(tr);
  });
}

function renderDetail(r) {
  const history = entityHistory(r.entity_id);
  const entityContext = r.entity_context || {};
  const grounded = r.groundedness_passed === undefined
    ? '<span style="color:#91a4b8">not run — execute run_reasoning.py</span>'
    : (r.groundedness_passed ? '<span class="grounded-ok">passed ✓</span>' : '<span class="grounded-bad">F
AILED ✗</span>');

  const features = Array.from(new Set(r.contributing_features || []));
  const featureHtml = features.length
    ? features.map(f => `
        <div class="chip">
          <span class="raw">${f}</span>
          <span class="explain">${describeFeature(f)}</span>
        </div>
      `).join('')
    : '<div class="chip"><span class="explain">No contributing features available.</span></div>';

  document.getElementById('detail').innerHTML = `
```

```
    <h2>${r.session_id}</h2>
    <div class="detail-kicker">${entityContext.asset_type || r.asset_type || 'unknown'} asset in ${entityC
ontext.zone || r.zone || 'unknown'} zone · ${entityContext.alert_count ?? r.alert_count ?? 0} total alerts
 for this entity</div>

    <div class="grid">
      <div class="field compact">
        <label>Entity profile</label>
        <div class="entity-meta">
          <div class="metric-tile"><div class="k">Asset type</div><div class="v">${entityContext.asset_typ
e || r.asset_type || 'unknown'}</div></div>
          <div class="metric-tile"><div class="k">Zone</div><div class="v">${entityContext.zone || r.zone
|| 'unknown'}</div></div>
          <div class="metric-tile"><div class="k">Criticality</div><div class="v">${entityContext.critical
ity ?? r.asset_criticality}/5</div></div>
          <div class="metric-tile"><div class="k">Alert count</div><div class="v">${entityContext.alert_co
unt ?? r.alert_count ?? 0}</div></div>
        </div>
        <div class="val" style="margin-top:10px;color:#91a4b8">Home geo: ${entityContext.home_geo || r.ent
ity_home_geo || 'unknown'} · Home device: ${entityContext.home_device || 'unknown'}</div>
      </div>

      <div class="field compact">
        <label>Entity / Type / MITRE</label>
        <div class="val">${r.entity_id} — <strong>${r.anomaly_type}</strong> (confidence ${r.type_confiden
ce})</div>
        <div class="val" style="margin-top:8px">${r.mitre_technique_name || 'n/a'} ${r.mitre_technique_id
|| ''}</div>
        <div class="val" style="margin-top:8px;color:#91a4b8">Groundedness check: ${grounded}</div>
      </div>
    </div>

    <div class="grid">
      <div class="field">
        <label>Risk score</label>
        <div class="val">anomaly_score=${r.anomaly_score} · asset_criticality=${r.asset_criticality}/5 · <
strong>impact_score=${r.impact_score}</strong></div>
      </div>

      <div class="field">
        <label>Recommended action</label>
        <div class="val"><span class="badge ${r.recommended_action}">${r.recommended_action}</span></div>
      </div>
    </div>

    <div class="field" style="margin-bottom:14px;">
      <label>Contributing factors</label>
      <div class="chips">${featureHtml}</div>
    </div>

    <div class="field" style="margin-bottom:14px;">
      <label>Narrative (Layer 4)</label>
      <div class="val">${r.narrative_summary || '<em>not generated yet — run run_reasoning.py</em>'}</div>
      <div class="val" style="margin-top:8px;color:#91a4b8">${r.narrative_why_flagged || ''}</div>
    </div>

    <div class="field">
      <label>Entity history (${history.length} flagged session${history.length===1?'':'s'} for this entity
)</label>
      <div class="chips">${history.map(h => `<div class="chip"><span class="explain">${h.anomaly_type} · $
{h.impact_score.toFixed(2)}</span></div>`).join('')}</div>
    </div>

    <div class="actions">
      <button class="btn-ack ${r.recommended_action==='acknowledge'?'btn-active':''}" onclick="alert('Ackn
owledged (visual only — no backend action wired yet).')">Acknowledge</button>
      <button class="btn-iso ${r.recommended_action==='isolate'?'btn-active':''}" onclick="alert('Isolate
requested (visual only — no backend action wired yet).')">Isolate</button>
      <button class="btn-esc ${r.recommended_action==='escalate'?'btn-active':''}" onclick="alert('Escalat
ed (visual only — no backend action wired yet).')">Escalate</button>
    </div>
  `;
}

load();
</script>
```

```
</body>
</html>
```